

系统开发工具基础

第一周实验报告

姓名：Yang Kunwang

学号：25020007154

1 课堂

1.1 q01 含空格文件名批量处理

实验内容

```
创建input/docs 和 input/tmp
mkdir -p input/docs input/tmp
创建input/docs/notes one.txt (内容为alpha和beta两行)
printf 'alpha\nbeta\n' > 'input/docs/notes one.txt'
创建input/docs/.secret.txt (内容为 hidden)
echo 'hidden' > input/docs/.secret.txt
创建input/tmp/empty.txt (空文件)
touch input/tmp/empty.txt
创建input/run.log (任意两行日志)
printf 'start\nfinished\n' > input/run.log
```

显示q01的绝对路径

```
pwd
```

并以长格式列出input下全部项目，包含隐藏文件

```
ls -la input/
```

把input下所有.txt文件复制到work/<学号>/，保留相对目录结构并正确处理名称中的空格。

```
mkdir -p work/25020007154
```

```
(cd input && find . -name '*.txt' -exec cp --parents {} ../work/25020007154/ \;)
```

将复制后的目录权限设为750、普通文件权限设为640

```
find work/25020007154 -type d -exec chmod 750 {} \;
```

```
find work/25020007154 -type f -exec chmod 640 {} \;
```

生成inventory.txt，列出相对路径和字节数

```
(cd work/25020007154 && find . -type f -printf '%p %s\n') > inventory.txt
```

实验结果

```

ykw@ykw-VirtualBox:~/dev_tools_lab/week1/q01$ pwd
/home/ykw/dev_tools_lab/week1/q01
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/q01$ ls -la input/
total 20
drwxrwxr-x 4 ykw ykw 4096 Aug 24 15:15 .
drwxrwxr-x 4 ykw ykw 4096 Aug 27 16:47 ..
drwxrwxr-x 2 ykw ykw 4096 Aug 24 15:14 docs
-rw-rw-r-- 1 ykw ykw 15 Aug 24 15:15 run.log
drwxrwxr-x 2 ykw ykw 4096 Aug 24 15:15 tmp
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/q01$ find work/25020007154 -type f
work/25020007154/tmp/empty.txt
work/25020007154/docs/notes one.txt
work/25020007154/docs/.secret.txt
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/q01$ ls -la work/25020007154/docs/
total 16
drwxr-x--- 2 ykw ykw 4096 Aug 27 16:46 .
drwxr-x--- 4 ykw ykw 4096 Aug 27 16:46 ..
-rw-r----- 1 ykw ykw 7 Aug 27 16:46 .secret.txt
-rw-r----- 1 ykw ykw 11 Aug 27 16:46 'notes one.txt'
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/q01$ cat inventory.txt
./tmp/empty.txt 0
./docs/notes one.txt 11
./docs/.secret.txt 7
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/q01$

```

1.2 q02 日志数据分析脚本

实验内容

在q02中创建access.csv，写入日志数据

```

cat > access.csv <<'EOF'
timestamp,user,path,status,latency_ms
09:00:01,alice,/api/users,200,120
09:00:02,bob,/api/orders,500,310
09:00:03,alice,/api/users,503,180
09:00:04,carol,/login,500,90
09:00:05,bob,/api/orders,500,260
09:00:06,dave,/health,200,20
09:00:07,alice,/api/orders,500,420
09:00:08,carol,/api/users,500,150
EOF

```

创建 analyze.sh 脚本文件

```

cat > analyze.sh <<'EOF'
#!/bin/bash
if [ $# -ne 1 ];then
    echo "需要传入文件路径" >&2

```

```

        exit 1
    fi
    FILE="$1"
    if [ ! -f "$FILE" ];then
        echo "错误: 文件 $FILE 不存在" >&2
        exit 1
    fi

    echo "====5xx最多前2个path===="
    tail -n +2 "$FILE" | awk -F ',' '{ $4 >=500 && $4 <600 {print $3}}' | sort | uniq -c | sort -k2

    echo "====平均latency_ms===="
    tail -n +2 "$FILE" | awk -F ',' '{sum+=$5; cnt++} END{printf "%.2f\n", sum/cnt}'
EOF

```

添加脚本可执行权限

```
chmod +x analyze.sh
```

对access.csv运行脚本

```
./analyze.sh access.csv
```

测试不存在的文件，验证报错

```
./analyze.sh not_exist.csv
```

实验结果

```

ykw@ykw-VirtualBox:~/dev_tools_lab/week1/q02$ ./analyze.sh access.csv
====5xx最多前2个path====
 3 /api/orders
 2 /api/users
====平均latency_ms====
193.75
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/q02$ ./analyze.sh not_exist.csv
错误: 文件 not_exist.csv 不存在
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/q02$ echo $?
1
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/q02$ tail -n +2 access.csv | awk -F ',' '{ $4 >=500 && $4 < 600 {print $3}}' | sort | uniq -c | sort -k1,1nr -k2
,2 | head -n 2
 3 /api/orders
 2 /api/users
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/q02$ tail -n +2 access.csv | awk -F ',' '{sum+=$5; cnt++} END{printf "%.2f\n", sum/cnt}'
193.75
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/q02$

```

1.3 q03 Git 分支合并冲突实验

实验内容

初始化仓库，在main创建config.txt，内容为mode=normal，并完成初始提交

```
git init
```

```
echo 'mode=normal' > config.txt
```

```
git add config.txt
```

```
git commit -m "init: mode=normal"
```

创建feature-a分支，把内容改为mode=safe并提交

```
git checkout -b feature-a
echo 'mode=safe' > config.txt
git add config.txt
git commit -m "feature-a: set mode=safe"
```

切回初始main分支，从初始main创建feature-b，把内容改为mode=fast并提交

```
git checkout main
git checkout -b feature-b
echo 'mode=fast' > config.txt
git add config.txt
git commit -m "feature-b: set mode=fast"
```

回到main，先合并feature-a

```
git checkout main
git merge feature-a
```

再合并feature-b，触发文件合并冲突

```
git merge feature-b
```

手动编辑config.txt解决冲突，保留mode=safe，并另加一行note=reviewed
config.txt最终文件内容：

```
mode=safe
note=reviewed
```

将冲突解决后的文件加入暂存区，完成合并提交

```
git add config.txt
git commit -m "merge: resolve conflict, keep safe, add note"
```

确认工作区干净，查看完整分支提交历史

```
git status
git log --all --graph --decorate --oneline
```

实验结果

```

ykw@ykw-VirtualBox:~/dev_tools_lab/week1/q03$ git status
位于分支 main
无文件要提交，干净的工作区
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/q03$ cat config.txt
mode=safe
note=reviewed
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/q03$ git log --all --graph --decorate --oneline
* 061f966 (HEAD -> main) resolve merge conflict
| \
| * d2e0886 (feature-b) feature-b: mode=fast
* | b40b260 (feature-a) feature-a: mode=safe
| /
* 01d922f init: mode=normal
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/q03$

```

1.4 q04 LaTeX 文档构建

实验内容

编写 report.tex 完整源码

vim report.tex

```

\documentclass{article}
\usepackage{amsmath}
\usepackage{booktabs}
\begin{document}
\title{Tool Report}
\author{ykw--25020007154}
\maketitle
\section{Result}

```

The calculation formula is given below, as shown in Equation~\eqref{eq:speed}:

```

\begin{equation}
v = \frac{s}{t}
\label{eq:speed}
\end{equation}

```

Equation~\eqref{eq:speed} represents the relationship among speed, distance, and time.

The experimental data is shown in Table~\ref{tab:data}.

```

\begin{table}[htbp]
\centering
\caption{Experimental Data Table}
\label{tab:data}
\begin{tabular}{cc}
\toprule
Distance(m) & Time(s) \\

```

```

\midrule
10.0 & 2.5 \\
20.0 & 5.0 \\
\bottomrule
\end{tabular}
\end{table}

```

```

\end{document}

```

使用`latexmk`编译生成pdf

```

latexmk -pdf -halt-on-error report.tex

```

#查看生成结果

```

ls -la

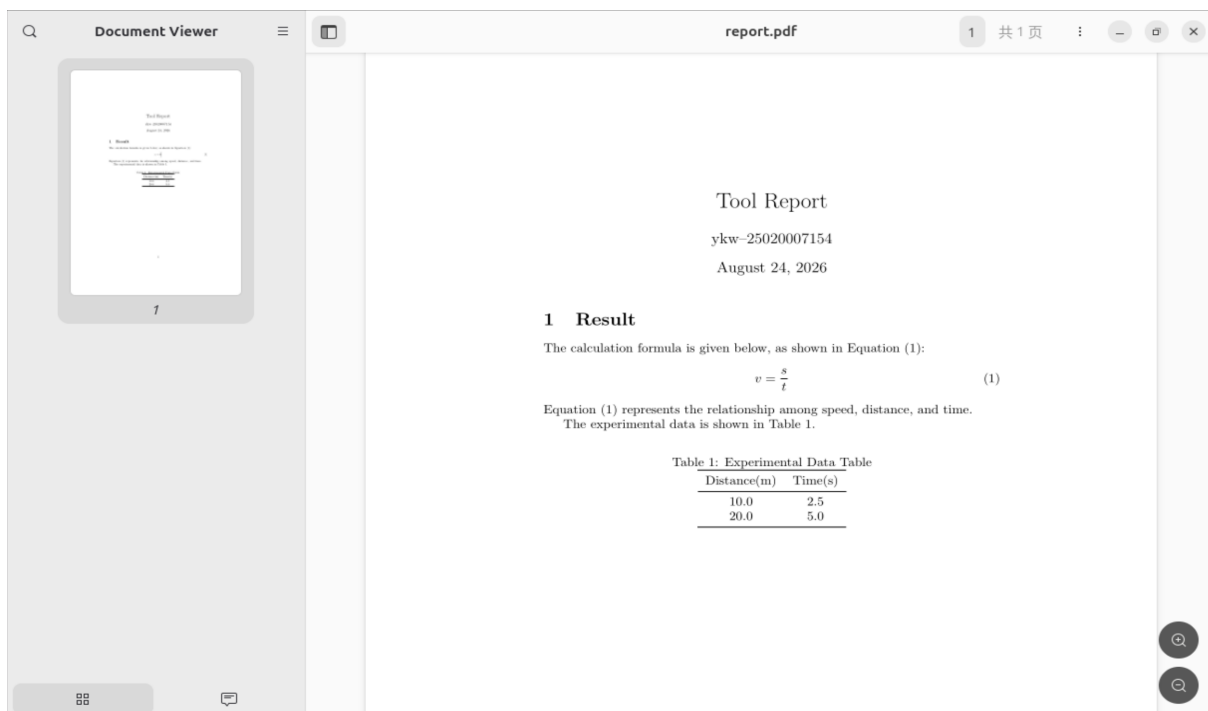
```

实验结果

```

ykw@ykw-VirtualBox:~/dev_tools_lab/week1/q04$ ls -la
total 92
drwxrwxr-x 2 ykw ykw 4096 Aug 30 16:58 .
drwxrwxr-x 6 ykw ykw 4096 Aug 26 17:42 ..
-rw-rw-r-- 1 ykw ykw 336 Aug 24 19:19 report.aux
-rw-rw-r-- 1 ykw ykw 4288 Aug 24 19:19 report.fdb_latexmk
-rw-rw-r-- 1 ykw ykw 3279 Aug 24 19:19 report.fls
-rw-rw-r-- 1 ykw ykw 5888 Aug 24 19:19 report.log
-rw-rw-r-- 1 ykw ykw 55975 Aug 24 19:19 report.pdf
-rw-rw-r-- 1 ykw ykw 709 Aug 24 19:18 report.tex
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/q04$

```



2 课后

2.1 练习 2

实验内容

弄清 `ls -l` 中 `-l` 参数的作用

在终端执行 `ls -l /`，查看根目录长格式输出结果

实验结果

执行 `ls -l` 后，输出每行开头的 10 个字符为文件权限标识。

第一个字符代表文件类型：`d` 表示目录，`-` 表示普通文件，`l` 代表软链接。

之后 9 个字符分为三组，依次对应所有者、所属用户组、其他用户的访问权限。

每组 3 位，`r` 为可读，`w` 为可写，`x` 为可执行；`-` 表示该项权限未开启。

以输出结果 `drwxr-xr-x` 举例：

该对象是目录；

所有者拥有可读、可写、可执行权限；

组用户与其他用户仅有可读、可执行权限，没有写入权限。


```

ykw@ykw-VirtualBox:~$ man ls
ykw@ykw-VirtualBox:~$ ls -l /
\total 3115084
lrwxrwxrwx    1 root root          7 Oct  3  2025 bin -> usr/bin
drwxr-xr-x    4 root root      4096 Aug 24 16:42 boot
dr-xr-xr-x    2 root root      4096 Oct  7  2025 cdrom
drwxr-xr-x   19 root root      4360 Aug 31 11:07 dev
drwxr-xr-x  151 root root     12288 Aug 24 19:00 etc
drwxr-xr-x    3 root root      4096 Oct 17  2025 home
lrwxrwxrwx    1 root root          7 Oct  3  2025 lib -> usr/lib
lrwxrwxrwx    1 root root          9 Oct  3  2025 lib64 -> usr/lib64
drwx-----   2 root root     16384 Oct 17  2025 lost+found
drwxr-xr-x    3 root root      4096 Oct 17  2025 media
drwxr-xr-x    2 root root      4096 Oct  7  2025 mnt
drwxr-xr-x    2 root root      4096 Oct  7  2025 opt
dr-xr-xr-x  285 root root          0 Aug 31 11:07 proc
drwx-----   7 root root      4096 Aug 30 14:53 root
drwxr-xr-x   40 root root        940 Aug 31 11:11 run
lrwxrwxrwx    1 root root          8 Oct  3  2025 sbin -> usr/sbin
drwxr-xr-x   17 root root      4096 Aug 24 14:58 snap
drwxr-xr-x    2 root root      4096 Oct  7  2025 srv
-rw-----   1 root root 3189768192 Oct 17  2025 swap.img
dr-xr-xr-x   13 root root          0 Aug 31 11:07 sys
drwxrwxrwt   17 root root        380 Aug 31 11:32 tmp
drwxr-xr-x   12 root root      4096 Oct  7  2025 usr
drwxr-xr-x   14 root root      4096 Oct 17  2025 var
ykw@ykw-VirtualBox:~$ \

```

2.2 练习 3

实验内容

理解命令 `find ~/Downloads -type f -name "*.zip" -mtime +30`，认识其中的 glob 模式 `*.zip`，创建测试目录，生成若干测试文件

```
mkdir t03
```

```
cd t03
```

```
touch 1.txt note.txt file1.txt fileA.txt a.txt b.txt c.txt
```

依次执行三条通配符测试命令

```
ls *.txt
```

```
ls file?.txt
```

```
ls {a,b,c}.txt
```

实验结果

`ls *.txt` 中 `*` 可匹配任意长度的字符，因此列出目录下所有后缀为 `.txt` 的文件。

`ls file?.txt` 中 `?` 仅匹配**单个任意字符**，只会匹配 `file` 后紧跟一个字符的文件。

`ls {a,b,c}.txt` 为大括号展开，等价于 `ls a.txt b.txt c.txt`，一次性生成三个文件名。

在 `find` 命令 `-name "*.zip"` 中，双引号阻止了 Shell 提前展开 glob，将 `*.zip` 原样传递给 `find` 和

```
ykw@ykw-VirtualBox:~$ cd dev_tools_lab
ykw@ykw-VirtualBox:~/dev_tools_lab$ cd week1
ykw@ykw-VirtualBox:~/dev_tools_lab/week1$ mkdir t03
ykw@ykw-VirtualBox:~/dev_tools_lab/week1$ cd t03
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t03$ touch 1.txt note.txt file1.txt fileA.txt a.txt b.txt c.txt
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t03$ ls *.txt
1.txt a.txt b.txt c.txt file1.txt fileA.txt note.txt
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t03$ ls file?.txt
file1.txt fileA.txt
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t03$ ls {a,b,c}.txt
a.txt b.txt c.txt
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t03$
```

2.3 练习 4

实验内容

测试命令：

```
echo 'Price:$100 !'
```

```
echo "Price:$100 !"
```

```
echo '$'Price:$100 !\nSecond Line'
```

实验结果

单引号将内部字符串完全字面输出，`\$`和`!`不会被 Shell 解析处理。

双引号会识别`\$`符号，尝试进行变量展开；交互式环境中`!`会触发命令历史扩展。

`\$` ``引号可以识别`\n`换行转义字符，实现字符串换行输出，`\$`与`!`以字面形式打印。

```
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t03$ echo 'Price:$100 !'
Price:$100 !
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t03$ echo "Price:$100 !"
Price:00 !
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t03$ echo '$'Price:$100 !\nSecond Line'
Price:$100 !
Second Line
```

2.4 练习 5

实验内容

执行两条重定向命令，分别实现标准输出与标准错误分离保存、合并保存：

```
ls /nonexistent /tmp > out.txt 2> err.txt
```

```
ls /nonexistent /tmp > all.txt 2>&1
```

查看结果命令

```
cat out.txt
```

```
cat err.txt
```

```
cat all.txt
```

实验结果

两条重定向命令执行完成后终端没有任何打印输出，所有内容都被写入文件。

```

ykw@ykw-VirtualBox:~$ ls /nonexistent /tmp > out.txt 2> err.txt
ykw@ykw-VirtualBox:~$ ls /nonexistent /tmp > all.txt 2>&1
ykw@ykw-VirtualBox:~$ cat out.txt
/tmp:
VMwareDnD
snap-private-tmp
systemd-private-25e6a3f3ce2d411eb69a2d44e5356e81-ModemManager.service-a3bw94
systemd-private-25e6a3f3ce2d411eb69a2d44e5356e81-chrony.service-a88X3p
systemd-private-25e6a3f3ce2d411eb69a2d44e5356e81-colord.service-5JIhPf
systemd-private-25e6a3f3ce2d411eb69a2d44e5356e81-fwupd.service-Zdkgmd
systemd-private-25e6a3f3ce2d411eb69a2d44e5356e81-polkit.service-EkGtKl
systemd-private-25e6a3f3ce2d411eb69a2d44e5356e81-power-profiles-daemon.service-iZCuQ8
systemd-private-25e6a3f3ce2d411eb69a2d44e5356e81-switcheroo-control.service-C5wxbK
systemd-private-25e6a3f3ce2d411eb69a2d44e5356e81-systemd-logind.service-j3E996
systemd-private-25e6a3f3ce2d411eb69a2d44e5356e81-upower.service-dgjBEj
ykw@ykw-VirtualBox:~$ cat err.txt
ls: cannot access '/nonexistent': No such file or directory
ykw@ykw-VirtualBox:~$ cat all.txt
ls: cannot access '/nonexistent': No such file or directory
/tmp:
VMwareDnD
snap-private-tmp
systemd-private-25e6a3f3ce2d411eb69a2d44e5356e81-ModemManager.service-a3bw94
systemd-private-25e6a3f3ce2d411eb69a2d44e5356e81-chrony.service-a88X3p
systemd-private-25e6a3f3ce2d411eb69a2d44e5356e81-colord.service-5JIhPf
systemd-private-25e6a3f3ce2d411eb69a2d44e5356e81-fwupd.service-Zdkgmd
systemd-private-25e6a3f3ce2d411eb69a2d44e5356e81-polkit.service-EkGtKl
systemd-private-25e6a3f3ce2d411eb69a2d44e5356e81-power-profiles-daemon.service-iZCuQ8
systemd-private-25e6a3f3ce2d411eb69a2d44e5356e81-switcheroo-control.service-C5wxbK
systemd-private-25e6a3f3ce2d411eb69a2d44e5356e81-systemd-logind.service-j3E996
systemd-private-25e6a3f3ce2d411eb69a2d44e5356e81-upower.service-dgjBEj

```

打开 `out.txt`，里面存放 `/tmp` 目录列表，属于标准输出（stdout 文件描述符 1）；

打开 `err.txt`，里面存放找不到 `/nonexistent` 的报错信息，属于标准错误（stderr 文件描述符 2）

二者被分开保存，证明 stdout 与 stderr 是两条互相独立的输出通道。

第二条命令中 `> all.txt` 将标准输出重定向写入文件；

`2>&1` 把标准错误的输出目标复制为标准输出的目标，

因此正常信息与报错信息都会写入 `all.txt`，实现两类数据流合并。

2.5 练习 6

实验内容

编写单行命令，仅当 `/tmp/mydir` 目录不存在时创建该目录：

```
test ! -d /tmp/mydir && mkdir /tmp/mydir
```

实验结果

`test ! -d /tmp/mydir`：检测目录是否存在；

目录不存在时命令返回成功（退出状态 \$?=0），存在则返回失败。

`&&` 为逻辑与运算符，只有前面命令执行成功，后面 `mkdir` 创建目录的命令才会运行。

如果 `/tmp/mydir` 已经存在，前面检测命令失败，`&&` 后面的创建命令不会执行，避免抛出目录已存

```
ykw@ykw-VirtualBox:~$ test ! -d /tmp/mydir && mkdir /tmp/mydir
ykw@ykw-VirtualBox:~$ ls -ld /tmp/mydir
drwxrwxr-x 2 ykw ykw 40 Aug 31 14:00 /tmp/mydir
ykw@ykw-VirtualBox:~$
```

2.6 练习 8

实验内容

使用 Vim 在 t08 目录创建脚本文件 checkfile.sh

```
vim checkfile.sh
```

按下`i`进入插入模式，写入 Shell 脚本

```
#!/bin/bash
if [ -f "$1" ]; then
    echo "文件 $1 存在"
else
    echo "文件 $1 不存在"
fi
```

按下`Esc`，输入`:wq`保存退出

添加脚本可执行权限

```
chmod +x checkfile.sh
```

运行脚本，传入文件路径参数进行两组测试

```
./checkfile.sh ../q01/input/run.log
```

```
./checkfile.sh fake.txt
```

实验结果

```
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t08$ ./checkfile.sh ~/dev_tools_lab/week1/q01/input/run.log
文件 /home/ykw/dev_tools_lab/week1/q01/input/run.log 存在
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t08$ ./checkfile.sh fake.txt
文件 fake.txt 不存在
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t08$
```

2.7 练习 9

实验内容

使用 Vim 创建脚本文件 check.sh，写入和上一题完全一致的脚本代码

```
vim check.sh
```

按下 i 进入插入模式，输入脚本：

```
#!/bin/bash
if [-f "$1"]; then
echo "文件 $1 存在"
```

```
else
echo "文件 $1 不存在"
fi
```

按下 **Esc**，输入 **:wq** 保存退出。

查看脚本初始权限

```
ls -l check.sh
```

未添加执行权限，直接尝试运行脚本

```
./check.sh somefile
```

给脚本增加可执行权限

```
chmod +x check.sh
```

再次查看权限，对比前后变化

```
ls -l check.sh
```

添加权限后，重新运行脚本测试

```
./check.sh somefile
```

实验结果



```
ykw@ykw-VirtualBox:~/dev_tools_lab/week1$ mkdir t09
ykw@ykw-VirtualBox:~/dev_tools_lab/week1$ cd t09
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t09$ vim check.sh
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t09$ ls -l check.sh
-rw-rw-r-- 1 ykw ykw 101 Aug 31 14:59 check.sh
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t09$ ./check.sh somefile
bash: ./check.sh: 权限不够
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t09$ chmod +x check.sh
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t09$ ls -l check.sh
-rwxrwxr-x 1 ykw ykw 101 Aug 31 14:59 check.sh
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t09$ ./check.sh somefile
文件 somefile 不存在
```

2.8 练习 10

实验内容

创建测试脚本 `trace.sh`

```
vim trace.sh
```

写入脚本内容：

```
#!/bin/bash
```

```
set -x
```

```
echo "开始执行脚本"
```

```
a=10
```

```
b=20
```

```
echo $((a+b))
```

添加执行权限

```
chmod +x trace.sh
```

运行脚本

```
./trace.sh
```

实验结果

`set -x` 开启了 `shell` 的执行跟踪模式。

开启后，脚本每一条被执行的命令，都会先打印出来，输出行前面带有 `+` 标记。

它会吧变量展开之后的真实命令展示在终端。

作用：脚本调试，查看代码实际执行了什么指令，定位出错位置。

```
ykw@ykw-VirtualBox:~/dev_tools_lab/week1$ mkdir t010
ykw@ykw-VirtualBox:~/dev_tools_lab/week1$ cd t010
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t010$ vim trace.sh
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t010$ chmod +x trace.sh
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t010$ ./trace.sh
+ echo 开始执行脚本
开始执行脚本
+ a=10
+ b=20
+ echo 30
30
```

2.9 练习 11

实验内容

创建测试文件

```
touch notes.txt
```

执行带日期备份命令

```
cp notes.txt notes_$(date +%Y-%m-%d).txt
```

查看生成结果

```
ls -la
```

实验结果

```
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t011$ cp notes.txt notes_$(date +%Y-%m-%d).txt
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t011$ ls -la
total 8
drwxrwxr-x  2 ykw ykw 4096 Aug 31 15:42 .
drwxrwxr-x 11 ykw ykw 4096 Aug 31 15:35 ..
-rw-rw-r--  1 ykw ykw   0 Aug 31 15:37 notes.txt
-rw-rw-r--  1 ykw ykw   0 Aug 31 15:42 notes_2026-08-31.txt
ykw@ykw-VirtualBox:~/dev_tools_lab/week1/t011$
```

3 附录

github 链接: https://github.com/mimimi67/dev_tools_lab.git

commit 记录截图:

```
ykw@ykw-VirtualBox:~/dev_tools_lab/week1$ git log
commit a9b2f39de39df7c9ad316704412d8a67e803def0 (HEAD -> main, origin/main)
Author: mimimi67 <1873904077@qq.com>
Date: Mon Aug 31 20:15:37 2026 +0800

    提交课后练习

commit d8ee475be65279de6ca70ba50ba747e1ffee30b7
Author: mimimi67 <1873904077@qq.com>
Date: Mon Aug 31 11:10:56 2026 +0800

    fix: 更新q01 q02

commit 3b867d786a70afc2738538f0785f63dfb991a743
Author: mimimi67 <1873904077@qq.com>
Date: Wed Aug 26 17:50:48 2026 +0800

    week1-commit4: finish q04

commit 395e3cd5043f514555c302ee511c3fff7cc08644
Author: mimimi67 <1873904077@qq.com>
Date: Wed Aug 26 17:48:07 2026 +0800

    week1-commit3: finish q03

commit d17db49e0f601833942fd5b055960ecba618fe
Author: mimimi67 <1873904077@qq.com>
Date: Wed Aug 26 17:46:36 2026 +0800

    week1-commit2: finish q02
```